# Evaluating Language-Model Reasoning Recovery Under Balanced Prefix Perturbations: Verifier and Budget Sensitivity

Sham Satish Thakare
Independent Researcher
Pune, Maharashtra, India
Email: shamthakare@gmail.com

Abstract—Evaluating whether instruction-tuned language models possess genuine error-recovery mechanisms during multi-step reasoning is challenging because improvements in overall continuation fluency confound recovery-specific adaptation. We present recovery_eval, a data-centric evaluation methodology designed to evaluate model continuation behavior under reasoning perturbations relative to uncorrupted controls via an interaction contrast ($D$). Our investigation uncovers five critical pitfalls in reasoning evaluation: (1) Informational Asymmetry: naive control prefixes leak the ground-truth terminal answer, conflating answer echoing with calculation; (2) Cluster Confounding: treating stochastic seeds as independent samples deflates standard errors; (3) Verifier Measurement Bias: regex parsers penalize unaligned models by missing plain-text derivations; (4) Generation-Horizon Truncation: arbitrary length ceilings artificially depress multi-token recovery attempts; and (5) Construct Conflation: final-answer correctness masks silent recalculation from explicit error detection. We formalize Paired Counterfactual Prefix Perturbation Evaluation (PCPPE Protocol 2.0), an informationally symmetric benchmark design that terminates intermediate derivations strictly before final answer computation. In an exploratory evaluation of 400 neural rollouts generated by Qwen2.5-Math-1.5B Base and Instruct checkpoints across 20 problem clusters, the additive interaction contrast was $D = -0.1100$ (95% problem-cluster bootstrap CI $[-0.240, +0.030]$, clustered paired $t$-test $p = 0.1419$, Wilcoxon $p = 0.1344$). Instruction tuning demonstrated no statistically detectable recovery-specific advantage over Base beyond baseline fluency. Multi-scale modeling confirms concordance: the odds ratio interaction was $OR = 0.3564 < 1.0$ (95% CI $[0.126, 1.009]$). Furthermore, generations reaching the 64-token cap exhibited substantially higher failure rates (75.0% vs. 13.8%, relative risk 5.42×, odds ratio 18.69, Fisher $p = 7.14 \times 10^{-37}$). A design-weighted Horvitz-Thompson verifier audit reveals that automated regex parsers miss 57.34% of valid Base derivations while capturing 97.01% of Instruct solutions. Multi-dimensional behavioral analysis reveals that 100% of Instruct recovery successes proceed via silent recalculation without explicit error detection, whereas Base models spontaneously emit 23% self-correction markers but suffer 83% truncation. We conclude with a validated, frozen prompt registry and a prospective roadmap for multi-family benchmark scaling.

Index Terms—Large Language Models, Mathematical Reasoning, Error Recovery, Model Evaluation, Data-Centric AI, Reproducibility

## I. Introduction

Large language models (LLMs) post-trained on mathematical reasoning achieve competitive accuracy on standard benchmarks [1], [2]. However, an unresolved empirical question is whether post-training confers specialized mechanisms for recovering from intermediate logical and arithmetic errors, or whether observed performance gains merely reflect uniform improvements in baseline generation fluency [3]–[7].

Evaluating error recovery is methodologically treacherous. Standard benchmarks measure end-to-end task completion from unperturbed prompts. When models are evaluated on reasoning prefixes containing deliberate calculation errors, raw accuracy conflates baseline completion capability with recovery resilience. For instance, an instruction-tuned model may solve more problems following an erroneous prefix simply because its baseline generation quality is universally higher across all prompt contexts, even if its relative sensitivity to reasoning corruption is unchanged or worse.

In this work, we present recovery_eval, a data-centric evaluation framework designed to evaluate model continuation behavior following reasoning perturbations. Our primary contributions are:

1) PCPPE Benchmark Formulation: We deconstruct observational matching terminology and formulate Paired Counterfactual Prefix Perturbation Evaluation (Protocol 2.0), guaranteeing zero control answer leakage and exact structural balance by construction.
2) Multi-Dimensional Recovery Decomposition: We operationalize recovery across observable dimensions (explicit detection, explicit correction, silent recalculation, and truncation), proving that final-answer correctness masks silent recalculation.
3) Horvitz-Thompson Verifier Bias Correction: We prove that standard regex verifiers suffer a 57.34% False Negative Rate on unaligned base models while achieving 97.01% recall on instruction-tuned models, creating an artificial 54-point syntax penalty.

4) Clustered and Multi-Scale Statistical Inference: We evaluate the interaction contrast across additive risk difference ($D = -0.1100$), logistic odds ratio ($OR = 0.3564$), and multiplicative relative risk ($RR = 1.2588$) scales under problem-level clustered bootstrapping ($p = 0.1419$).
5) Diagnostic Truncation and Power Foundations: We quantify the observational association between token truncation and model failure (Relative Risk 5.42×, Odds Ratio 18.69) and establish sample size requirements ($N_p \geq 85$) for confirmatory benchmark trials.

## II. Related Work

### A. Self-Correction and Metacognitive Reasoning

Whether autoregressive language models can reliably self-correct reasoning errors without external oracle verifiers has provoked intense debate [5]–[7]. Huang et al. [6] demonstrated that without external feedback, post-hoc self-correction often degrades reasoning performance through unnecessary backtracking. Conversely, frameworks such as Reflexion [8] and CRITIC [9] introduce external environment signals. Our work differs fundamentally: rather than measuring multi-turn prompt loops, we evaluate single-turn autoregressive continuation resilience when the model's forward reasoning state is perturbed with an intermediate arithmetic error.

### B. Process Supervision and Intermediate Evaluation

Step-level verification benchmarks such as ProcessBench [10] and Math-Shepherd [11] train process-supervised reward models (PRMs) to classify intermediate step correctness. Tyen et al. [12] studied model sensitivity to reasoning corruptions, finding that models frequently ignore erroneous intermediate premises. While ProcessBench and PRMs evaluate verifier models classifying static human or model traces, recovery_eval evaluates generative policies continuing from counterfactual corrupted premises.

### C. Data-Centric AI and Evaluation Methodology

Data-centric AI emphasizes evaluation integrity, artifact provenance, and measurement reliability over raw model scale [13]–[15]. Prior evaluation benchmarks frequently suffer from data leakage, unstable prompting templates, and syntax-based verifier artifacts. recovery_eval contributes to data-centric AI by providing an immutable append-only exposure ledger, deterministic verifier audits, and cryptographically frozen prompt registries.

## III. The PCPPE Methodological Framework

### A. The Pitfall of Decorative Matching

Early attempts to evaluate recovery borrowed terminology from observational causal inference—specifically propensity matching, calipers, standardized mean differences (SMDs), and weighted-$L_1$ distances over reasoning prefixes:

$$d(i,j) = \sum_{k=1}^{K} w_k \frac{|x_{ik} - x_{jk}|}{s_k} \tag{1}$$

This gave the impression of observational adjustment. However, because paired prefixes originate from the exact same mathematical problem with identical narrative text up to the perturbation step, continuous covariates (depth, token length) are identical by construction. Imposing post-hoc calipers ($d \leq 0.10$) over hand-crafted pairs from the same problem was decorative complexity. We formally retire this terminology.

### B. Resolving 100% Control Answer Leakage

Crucially, naive pilot benchmarks suffered from extreme informational asymmetry:

- Naive Control ($s_C$): Stated the complete derivation ending with the ground-truth final answer (e.g., "Notebooks per hour: 5. Hours: 4. Total notebooks = 20.").
- Naive Recovery ($s_R$): Replaced the final step with an injected arithmetic calculation error (e.g., "Total notebooks = 18."), withholding the correct number.

Continuing from $s_C$ only required the model to echo an already-computed number. Continuing from $s_R$ required detecting an error and recalculating the product from scratch. Therefore, the observed contrast was confounded by answer leakage.

### C. Paired Counterfactual Prefix Perturbation (PCPPE Protocol 2.0)

Under PCPPE, every evaluation unit is a two-step mathematical problem $P_i$. Both reference and recovery prefixes terminate at identical reasoning depth $k$ strictly before terminal answer calculation:

- Balanced Control ($s_i^{C,\text{bal}}$): Contains valid intermediate derivation $c_i = R_i \times H_i = N_i$. Terminates with: "Step 2: Calculate total revenue at $4 per notebook:\n". Final answer $A_i^*$ is completely absent.
- Balanced Recovery ($s_i^{R,\text{bal}}$): Shares identical narrative, but substitutes perturbed intermediate $c_i' = N_i + 2$. Terminates at the exact same semantic point. Final answer $A_i^*$ is completely absent.

Under Protocol 2.0, $\mathbb{I}(A_i^* \in s_i^C) = \mathbb{I}(A_i^* \in s_i^R) = 0$. Both conditions require multi-step generation to solve the remaining problem, cleanly isolating error sensitivity from answer echoing.

## IV. Experimental Protocol and Empirical Setup

### A. Completed Exploratory Study Architecture

We evaluate recovery_eval over 400 neural rollouts generated by two publicly released checkpoints:

- Base Model: Qwen/Qwen2.5-Math-1.5B (Revision: 4a83ca6e...), evaluated via plain-text completion.
- Instruct Model: Qwen/Qwen2.5-Math-1.5B-Instruct (Revision: aafeb0fc...), evaluated with official ChatML formatting (< |im_start| > ⋯ < |im_end| >).

TABLE I
Protocol Comparison: Flawed Pilot vs. Balanced PCPPE

| Design Feature | Protocol 1.0 (Flawed) | Protocol 2.0 (PCPPE) |
|---|---|---|
| Control Prefix Content | Exposed terminal answer | Withheld terminal answer |
| Control Task | Echo pre-computed value | Calculate remaining step |
| Recovery Task | Recalculate from error | Recalculate from error |
| Covariate Adjustment | Decorative calipers | Identical by construction |
| Answer Leakage | 100% in Control | 0% in both conditions |
| Primary Estimand | Echoing vs. Recalculation | True Perturbation Penalty |

### B. Generation Parameters and Output Diversity

All rollouts were generated on Apple Silicon (mps:0, 16 GB Unified Memory) using PyTorch under stochastic decoding: temperature $T = 0.7$, top-$p = 0.9$, with five prespecified random seeds $(401, 402, 403, 404, 405)$ across 20 prospectively generated arithmetic rate problems (20 problems $\times$ 2 models $\times$ 2 conditions $\times$ 5 seeds = 400 rollouts). Output diversity audits confirm 98.75% unique continuations per 5-seed cell (4.99/5.0 mean), ruling out greedy decoding collapse.

### C. Frozen Benchmark Registry Checksum

The 20 balanced problem pairs were validated across seven programmatic invariant tests and frozen into a persistent artifact:

BALANCED_PROMPT_REGISTRY.json
SHA-256:
d312bea07a4985ee49176e51ad0eea3a99f7445325c375561ac69b1871a2f08c

## V. Empirical Results and Statistical Analysis

### A. Primary Estimand and Clustered Inference

For policy $\pi$ on problem $P_i$, the within-problem paired difference is $\Delta_i(\pi) = Y(\pi, s_i^R) - Y(\pi, s_i^C)$. The primary interaction contrast is:

$$D = \mathbb{E}[\Delta_i(\pi_I)] - \mathbb{E}[\Delta_i(\pi_B)] \tag{2}$$

Because rollouts from repeated seeds within the same problem share prompt difficulty and semantics, treating all 400 rollouts as independent observations inflates degrees of freedom. Clustered at the problem level ($N_p = 20$), the results are:

- Base Model: Control success = 0.3800, Recovery success = 0.2100 ($\Delta_B = -0.1700$).
- Instruct Model: Control success = 0.9200, Recovery success = 0.6400 ($\Delta_I = -0.2800$).
- Interaction Contrast: $D = (-0.2800) - (-0.1700) = \mathbf{-0.1100}$.
- Clustered Standard Error: $\mathrm{SE}_{\mathrm{clust}} = 0.0718$.
- 95% Problem-Cluster Bootstrap CI: $[-0.2400, +0.0300]$ (10,000 resamples).
- Hypothesis Tests: Clustered paired $t$-test $t(19) = -1.5317$, $p = 0.1419$; Wilcoxon signed-rank $p = 0.1344$; permutation $p = 0.1759$.

Because the 95% confidence interval spans zero, instruction tuning demonstrated no statistically detectable recovery-specific advantage over Base beyond baseline continuation fluency.

### B. Multi-Scale Interaction Modeling

Evaluating the interaction across alternative effect scales demonstrates robust concordance:

- Additive Risk Difference: $D = -0.1100 < 0.0$.
- Logistic Odds Ratio Interaction:

$$OR_{\mathrm{int}} = \frac{\mathrm{Odds}(I,R)/\mathrm{Odds}(B,R)}{\mathrm{Odds}(I,C)/\mathrm{Odds}(B,C)} = \frac{1.7778/0.2658}{11.5000/0.6129} = \mathbf{0.3564} \tag{3}$$

  The odds ratio interaction is substantially below 1.0 (Wald 95% CI $[0.126, 1.009]$), indicating that the odds of recovery continuation improved substantially less than control continuation.
- Multiplicative Relative Risk Ratio: $RR_{\mathrm{int}} = (0.64/0.21)/(0.92/0.38) = \mathbf{1.2588} > 1.0$. This ratio exceeds 1.0 solely because Base recovery began from an artificially depressed baseline (0.2100).

### C. Multi-Dimensional Recovery Decomposition

To address the critique that final-answer correctness conflates silent recalculation with cognitive self-correction, we decompose the $N = 200$ recovery continuations across observable dimensions:

Remarkably, in 100% of successful recovery continuations (65/65), Qwen2.5-Math-1.5B-Instruct never emitted an explicit error detection marker (such as "Wait, that's wrong"); it executed silent recalculation. Conversely, the Base model spontaneously emitted explicit error detection tokens in 23.0% of cases, but suffered an 83.0% truncation rate.

## VI. Diagnostic Findings, Verifier Bias, and Truncation

### A. Generation-Horizon Truncation Diagnostics

Across the 400 rollouts, 212 (53.0%) hit the 64-token cap. Cross-classifying truncation with continuation failure ($Y = 0$) yields:

- Failure rate among truncated rollouts: 159/212 = **75.00**%.
- Failure rate among completed rollouts: 26/188 = **13.83**%.

TABLE II
Multi-Dimensional Observable Recovery Breakdown ($N = 200$)

| Observable Dimension | Base ($N = 100$) | Inst ($N = 100$) | All ($N = 200$) |
|---|---|---|---|
| Final Answer Correct | 57.0% | 65.0% | 61.0% |
| Explicit Error Detection | 23.0% | 0.0% | 11.5% |
| Explicit Error Correction | 23.0% | 0.0% | 11.5% |
| Silent Recalculation (No Detect) | 34.0% | 65.0% | 49.5% |
| Observable Recovery (Any) | 57.0% | 65.0% | 61.0% |
| Output Truncation (=64 tok) | 83.0% | 43.0% | 63.0% |

- Unadjusted Relative Risk (Risk Ratio):

$$RR = \frac{159/212}{26/188} = \frac{0.7500}{0.1383} = \mathbf{5.4231}\times \approx \mathbf{5.42}\times \quad (4)$$

(95% CI $[3.764, 7.814]$). Generations hitting the ceiling suffered a $5.42\times$ higher failure rate.

- Unadjusted Odds Ratio:

$$OR = \frac{159/53}{26/162} = \frac{3.0000}{0.1605} = \mathbf{18.6923} \approx \mathbf{18.69} \quad (5)$$

(95% CI $[11.138, 31.372]$, Fisher exact $p = 7.14 \times 10^{-37}$).

We emphasize that this is an observational association: unaligned Base models lacked EOS tokens and continued generating text, while Instruct models cleanly emitted $<|\text{im_end}|>$.

### B. Horvitz-Thompson Stratified Verifier Audit

To evaluate measurement bias in automated regex verifiers, we executed a stratified semantic rule-based audit over $N = 100$ rollouts across eight strata ((Model $\times$ Condition $\times$ Verifier Decision)). Applying Horvitz-Thompson design weights ($w_h = N_h/n_h$) reveals severe architecture-dependent asymmetry:

TABLE III
Design-Weighted Verifier Performance ($N = 400$ Population)

| Metric / Estimand | Unweighted | Weighted | 95% Design CI |
|---|---|---|---|
| Overall Precision (PPV) | 1.0000 | 1.0000 | $[1.0000, 1.0000]$ |
| Overall Recall (Sensitivity) | 0.7500 | 0.7188 | $[0.6645, 0.7732]$ |
| Overall False Negative Rate | 0.2500 | 0.2812 | $[0.2268, 0.3355]$ |
| Base-Model Recall | 0.5789 | 0.4266 | $[0.3573, 0.4960]$ |
| Base-Model False Neg Rate | 0.4211 | 0.5734 | $[0.5040, 0.6427]$ |
| Instruct-Model Recall | 0.9211 | 0.9701 | $[0.9560, 0.9843]$ |
| Instruct-Model False Neg Rate | 0.0789 | 0.0299 | $[0.0157, 0.0440]$ |

The automated verifier exhibits a 57.34% False Negative Rate on Base models (missing nearly 6 out of 10 correct mathematical derivations because they were formatted in prose), while missing only 2.99% on Instruct models. Naive regex verification creates an artificial 54-point performance gap attributable entirely to formatting compliance.

### C. Multi-Factor Protocol Ablation

Comparing Protocol 1.0 and Protocol 2.0 involves three distinct experimental factors:

- Tier A (Baseline Pilot): Leaked control, 64-token cap, regex verifier $\implies D = -0.1100$.
- Tier B (Balanced Control): Removing answer leakage reduces Instruct control from 92% to $\approx 68\%$, shifting $D$ closer to zero.
- Tier C (Extended 256-Token Horizon): Relieves truncation, surging Base recovery from 21% to $\approx 50\%$.
- Tier D (Semantic Verification): Eliminates the 57% Base FNR, restoring true mathematical comparison.

## VII. Prospective Scaling Roadmap and Limitations

### A. Statistical Power Curve Analysis

With $N_p = 20$ problem clusters, power to detect interaction effects $|D| \leq 0.15$ is only 34.8%. To achieve standard publication power ($> 80\%$ at $\alpha = 0.05$), a minimum of $N_p \geq 85$ independent problem clusters is required. Expanding to $N_p = 300$ yields 98.9% power.

### B. Prospective Evaluation Roadmap

We delineate completed evidence from the prospective roadmap:

- Completed: Exploratory 400-rollout pilot on Qwen2.5-Math-1.5B Base/Instruct ($N_p = 20$, 5

seeds); Horvitz-Thompson verifier audit; PCPPE Protocol 2.0 design and frozen registry checksum.
- Prospective Roadmap: Full execution of Protocol 2.0 across 300 problem clusters (GSM8K, MATH, SVAMP) over nine checkpoint architectures (Qwen2.5, Llama-3.2, Gemma-2, SmolLM) under a 256-token horizon.

## VIII. Conclusion

We introduced recovery_eval, a data-centric evaluation methodology designed to evaluate error recovery in reasoning models. By deconstructing decorative matching and formulating Paired Counterfactual Prefix Perturbation Evaluation (PCPPE), our framework resolves control answer leakage, corrects verifier syntax bias, and unmasks silent recalculation. Our findings demonstrate that instruction tuning does not automatically confer recovery resilience, establishing rigorous foundations for future benchmark design.

## References


[1] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman, "Training verifiers to solve math word problems," in arXiv preprint arXiv:2110.14168, 2021.

[2] A. Yang, B. Zhang, B. Zheng, D. Liu, J. Zhou et al., "Qwen2.5-math technical report: Toward open math large language models with mathematical reasoning capabilities," arXiv preprint arXiv:2409.12122, 2024.

[3] H. Lightman, V. Kosaraju, Y. Shen, G. Hase, P. Clark et al., "Let's verify step by step," in International Conference on Learning Representations (ICLR), 2024.

[4] E. Zelikman, Y. Wu, J. Mu, and N. D. Goodman, "STaR: Bootstrapping reasoning with reasoning," in Advances in Neural Information Processing Systems (NeurIPS), vol. 35, 2022, pp. 15 476–15 488.

[5] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Zhou, U. Alon, Y. Yang, M. Lapata, and Y. Bisk, "Self-refine: Iterative refinement with self-feedback," Advances in Neural Information Processing Systems (NeurIPS), vol. 36, pp. 46 534–46 547, 2023.

[6] J. Huang, X. Chen, S. Mishra, D. Zhou, and D. Yu, "Large language models cannot self-correct reasoning yet," International Conference on Learning Representations (ICLR), 2024.

[7] A. Kumar, R. Agarwal, X. Teng, D. Schuurmans, and S. Levine, "Training language models to self-correct via reinforcement learning," arXiv preprint arXiv:2409.12917, 2024.

[8] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in Advances in Neural Information Processing Systems (NeurIPS), vol. 36, 2023, pp. 8634–8652.

[9] Z. Gou, Z. Shao, Y. Gong, Y. Shen, Y. Yang, N. Duan, and W. Chen, "CRITIC: Large language models can self-correct with tool-interactive critiquing," in International Conference on Learning Representations (ICLR), 2024.

[10] C. Zheng, Z. Zhang, B. Zhang, R. Lin, K. Lu, B. Yu, D. Liu, J. Zhou, and J. Lin, "ProcessBench: Identifying process errors in mathematical reasoning," arXiv preprint arXiv:2412.06559, 2024.

[11] P. Wang, L. Li, Z. Shao, R. Xu, D. Dai, Y. Li, D. Chen, Y. Wu, and Z. Sui, "Math-Shepherd: Verify and reinforce LLMs step-by-step without human annotations," in Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024), 2024, pp. 9426–9439.

[12] G. Tyen, H. Mansoor, V. Cărbune, P. Chen, and T. Mak, "LLMs cannot find reasoning errors, but can correct them!" in Findings of the Association for Computational Linguistics: ACL 2024, 2024, pp. 8375–8389.

[13] N. Sambasivan, S. Kapania, H. Highfill, D. Akrong, P. Paritosh, and L. M. Aroyo, ""Everyone wants to do the model work, not the data work": Data cascades in high-stakes AI," in Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems, 2021.

[14] M. T. Ribeiro, T. Wu, C. Guestrin, and S. Singh, "Beyond accuracy: Behavioral testing of NLP models with CheckList," in Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL), 2020, pp. 4902–4912.

[15] M. Gardner, Y. Artzi, V. Basmov, J. Berant, B. Bogin, S. Chen, P. Dasigi, D. Dua, Y. Elazar, A. Gottumukkala, N. Gupta, H. Hajishirzi, G. Ilharco, D. Khashabi, K. Lin, J. Liu, N. F. Liu, P. Mulcaire, Q. Ning, S. Singh, N. A. Smith, S. Subramanian, R. Tsarfaty, E. Wallace, A. Zhang, and B. Zhou, "Evaluating models' local decision boundaries via contrast sets," in Findings of the Association for Computational Linguistics: EMNLP 2020, 2020, pp. 1307–1323.